FemboyEngine2D — Архитектурная спецификация

Языковой раздел: Русский

Version: 0.3.0-alpha.1

Status: Alpha

Date: 2026-11-05

Document: FBY-ARCH-0.1

---

1. Краткое резюме

FemboyEngine2D — двумерный движок на C++26, построенный вокруг data-oriented design и находящийся в статусе Release. Рантайм разделён на отдельные статические библиотеки по роли поставки: игра линкует движок, редактор — движок и библиотеку UI редактора, а офлайн-кукер линкует только библиотеки ассетов. В поставку игры не попадают Dear ImGui, ImGuizmo, ImPlot и TOML-парсер.

Правила проектирования намеренно жёсткие. Состояние симуляции — это простые trivially copyable данные; горячие пути используют фиксированную ёмкость, указатели на функции и арены; кадровые пути не выделяют память; каждая подсистема предоставляет детерминированное состояние с контрольной суммой. Именно это делает роллбэк, горячую перезагрузку, snapshot-тесты и GPU multi-draw практичными, а не теоретическими.

- Ядро: Definitions.hpp, Vector2D/3D/4D, jobs, платформа, plugin ABI, рефлексия Mirror.
- RHI: постоянные кольцевые буферы кадра, 48-байтные sprite-элементы, indirect-отрисовка мешей и один compute-диспатч на linear-blend skinning.
- ECS и физика: реестр EnTT, декларативный SystemGraph, Snapshot v4, мост Box2D v3, цепочки суставов, Verlet-ткань и две модели воды.
- Сеть: NetRollbackSession с кольцом снапшотов на 256 тиков, предсказанием ввода, повтором опоздавших входов и стабильными идентификаторами сущностей.
- Инструментарий: транзакционный CommandStack, AssetDatabase со строками .femeta, watcher, кукер, VFS и редакторы графов на ImNodes с AOT-транспилятором.

2. Иерархия репозитория и цели сборки

Весь исходный код находится в Source/. Корнем include-путей является сам Source, поэтому каждый include явно называет свой домен. Заголовок в Source/Engine/RHI

подключается как "Engine/RHI/RHI.hpp"; код редактора использует "Editor/Panels/...". Это убирает старый неявный префикс fby/ и делает принадлежность файла видимой с первой строки.

Путь	Детали
Source/Engine/Core	Определения, типы значений, jobs, платформа, плагины, Mirror
Source/Engine/RHI	Интерфейс рендеринга, шейдеры, материалы, стек камер
Source/Engine/ECS	Реестр, компоненты, сцены, снапшоты, скелет, анимация
Source/Engine/Input	Маппер действий, контексты, снимки геймпада, фиксированное состояние ввода
Source/Engine/Physics	Бэкенды Box2D v3 и DOD, соединения, транспорт, колёса, ткань, жидкость, частицы
Source/Engine/Audio	Микшер, DSP-граф, лимитер, Ogg/Opus
Source/Engine/Network	NetRollbackSession, кольца ввода, репликация снапшотов
Source/Engine/Assets	VFS, .femeta, кукер, локализация, форматы текстур/аудио
Source/Engine/Script	Байткод-VM, компилятор графов, AOT-бэкенд
Source/Engine/UI	PinkUI, гипертекст, атлас шрифтов, доступность
Source/Engine/Video	Воспроизведение и стриминг WebM/VP9
Source/Editor/Panels	Панели редактора и холсты ImNodes
Source/Editor/Infrastructure	Стек команд, watcher кукера, управление проектами
Source/Apps, Source/Tests, Source/Tools, Source/Plugins	Приложения, тесты, офлайн-инструменты, фикстуры плагинов

Имена файлов — PascalCase без исключений: Vector2D.hpp, NetRollbackSession.cpp, ComponentRegistry.cpp. Style gate отклоняет не-PascalCase имена, нижние подчёркивания, не-Allman скобки, комментарии в конце строки, кириллицу, dynamic_cast, typeid и любые оставшиеся идентификаторы Vec2, Vec3 или Vec4.

Цель	Детали
FemboyEngine2D::Engine	Рантайм-библиотека; без UI редактора и без TOML-парсера
FemboyEngine2D::Editor	Панели редактора, стек команд, интеграция ImGui/ImNodes
FemboyEngine2D::AssetCook	Кукинг текстур BC7/ASTC, общий для редактора и офлайн-инструментов
FemboyEngine2D::AssetLib	Компиляция паков и локализации; не поставляется с игрой
fby_shaderc_core	Обёртка glslang, переиспользуемая редактором в процессе
fby_sandbox, fby_editor, fby_* tools	Приложения, офлайн-кукеры, AOT-компилятор, scene tool

3. Базовые определения и система типов

Source/Engine/Core/Definitions.hpp — единственный источник истины для низкоуровневого словаря. Он определяет FORCEINLINE, помощник статического приведения FemCast<>, проверяемое полиморфное приведение Cast<>, стабильный

TypeId и базовый TypeObject. Он подключает Mirror и базовые типы значений, поэтому единица трансляции, включившая его, получает полный контракт без второго include.

```
using TypeId = std::uint64_t;
template <typename T> consteval TypeId StableTypeId() noexcept;

template <typename To, typename From>
[[nodiscard]] FORCEINLINE To FemCast(From&& InValue) noexcept;

template <typename To, typename From>
[[nodiscard]] FORCEINLINE To* Cast(From* InObject) noexcept;
```

StableTypeId<T>() хеширует имя компонента Mirror через FNV-1a. Поэтому отражённые компоненты имеют одну идентичность в инспекторе, схеме снапшотов, plugin ABI и контрольной сумме роллбэка. Класс, не являющийся компонентом, может закрепить независимое от компилятора имя через FBY_TYPE_NAME либо принять хеш сигнатуры компилятора. Cast<> выполняет upcast на этапе компиляции и обходит плоскую таблицу родителей при downcast; неверный downcast возвращает nullptr, а сгенерированная компилятором информация о типах не читается. dynamic_cast и typeid отсутствуют в исходниках движка, и style gate это проверяет.

Базовые типы значений trivially copyable и standard layout. Координатные типы — Vector2D, Vector3D и Vector4D с компонентами X, Y, Z и W в верхнем регистре. Rect — полуоткрытый axis-aligned прямоугольник; Color32 — один упакованный uint32 в порядке 0xAABBGGRR; Handle<Tag> — nullable 32-битный индекс пула, а не указатель. Каждый компонент проверяется компилятором на trivial copyability и standard layout до попадания в снапшот или таблицу плагина.

Слой	Детали
Jobs	Пул std::jthread, задачи на указателях функций, ParallelFor с участием вызывающего потока
Платформа	Окно/ввод SDL3, нативные диалоги, создание Vulkan surface
Плагины	C ABI, проверка layout hash, реестр предоставляется хостом
Mirror	consteval-дескрипторы компонентов из указателей на члены, без зависимости от P2996

4. RHI: Vulkan multi-draw и compute-скиннинг

RHI принимает FramePacket из простых данных и никогда не видит компонент, сущность или строку. Спрайт — один 48-байтный RenderItem: позиция, размер, uvMin, uvMax, слой, упакованный цвет, поворот и флаги. Элементы один раз за кадр сортируются 64-битным ключом (16 бит слой, 40 бит глубина, 8 бит последовательность) и загружаются в постоянно отображённое кольцо инстансов. GPU читает один элемент на инстанс; отдельного vertex-буфера, descriptor set или привязки текстуры на спрайт нет.

Путь мешей параллелен и намеренно устроен иначе. Статическая геометрия один раз загружается в device-local арену и адресуется через MeshHandle. Вершины

деформируемых SkeletalAnimation/SkinComponent потоково попадают в host-visible кольцо; один compute-диспатч выполняет linear-blend skinning для всего кадра; результат пишется в device-local буфер SkinnedVertex. MeshVertex занимает 40 байт, SkinnedVertex — 32 байта, обе стороны используют полную column-major Mat4, поэтому 2.5D-камера даёт настоящий z и перспективное деление.

```
MeshVertex     40 B  позиция, uv, цвет, skin range, индексы костей, веса
SkinnedVertex  32 B  однородная позиция (vec4), uv, цвет
RenderItem     48 B  позиция, размер, uvMin, uvMax, слой, цвет, поворот, флаги
```

Отрисовка выполняется одним vkCmdDrawIndexedIndirect на (texture array, слой, класс геометрии). Данные модели, слоя и материала на каждый draw лежат в storage-буфере, индексируемом gl_InstanceIndex, а firstInstance indirect-команды выбирает запись. Поэтому один multi-draw несёт разные model-матрицы без push constant на каждый меш. Vulkan-бэкенд требует shaderDrawParameters; null-бэкенд всё равно проходит полный путь FramePacket, чтобы тесты проверяли реальные данные, а не заглушку.

Бэкенд	**Детали**
Vulkan	Windows/Linux/Android; indirect-путь мешей, compute LBS, загрузчик в рантайме
Metal	macOS/iOS; тот же контракт FramePacket, argument buffers Metal
Null	Headless CI и серверы; полный путь пакетов/колец без устройства

По умолчанию в полёте три кадра. Каждый срез владеет кольцом инстансов, кольцом вершин мешей и кольцом индексов, выделенными один раз при создании устройства. Ёмкость — решение на старте, а не аллокация на кадр: значения по умолчанию покрывают 1 048 576 sprite-элементов, 65 536 вершин и 131 072 индекса мешей на срез.

Движковый `Rhi::FrameComposer` превращает один ECS-мир в этот пакет. Он отсекает спрайты по камере, складывает авторский цвет с результатом освещения, упаковывает motion vectors для временных апскейлеров, сортирует спрайты и tilemap-квады по ключу в батчи материалов и слоёв, строит OIT- и depth-prepass-разбиения и резервирует массивы от ёмкости устройства, поэтому ровный кадр не аллоцирует. Редактор и игра используют один класс; вьюпорт владеет одним композером, а пакет валиден до следующей композиции на этом композере.

5. ECS, планировщик и Snapshot v4

ECS — это реестр EnTT со строгим контрактом компонентов: trivially copyable, standard layout, без строк, контейнеров и vtable. Компоненты — данные; поведение живёт в системах. Кадр имеет четыре фазы: PreUpdate (переменный dt, ввод и стриминг), FixedUpdate (постоянный dt, геймплей и физика), PostUpdate (камера, куллинг, аудио) и Render (извлечение и редактор).

SystemContext, который получает система, несёт реестр и JobSystem напрямую, а остальные указатели группирует в доменные бандлы (render, assets, audio, physics, text, accessibility, gameplay); система читает только объявленные бандлы. Ядровые компоненты лежат в CoreComponents.hpp, поэтому модулям иерархии и трансформов не нужны заголовки текста и big-int. Запланированная система получает один из узких view (PhysicsSystemView, RenderSystemView, GameplaySystemView, AudioSystemView, UiSystemView, AccessibilitySystemView, CoreSystemView); Bind*System<&Function>() адаптирует его к ABI графа.

SystemGraph — производственный планировщик. Зависимости задаются явными битмасками after/before; однократная топологическая сортировка строит порядок; цикл или зависимость через фазы приводят к ошибке Build(). Система может объявить, какие сервисы движка она читает и пишет; две системы одной фазы с конфликтом по сервису упорядочиваются по регистрации, а не запускаются вместе. Граф вмещает до 64 узлов и использует указатели на функции плюс непрозрачный user-указатель, а не std::function. PluginHost удаляет все узлы с совпадающим user-указателем при выгрузке плагина и пересобирает скомпилированный порядок до следующего кадра.

Фаза	**Детали**
PreUpdate	Ввод, события окна, стриминг ассетов; переменный dt
FixedUpdate	Геймплей, Box2D, суставы, ткань, жидкость; 0..N шагов постоянного dt
PostUpdate	Слежение камеры, куллинг, отправка в микшер; переменный dt
Render	Извлечение рендера, сборка UI, отрисовка редактора; раз за кадр

Snapshot v4 — канонический образ байтов с контрольной суммой. 104-байтный заголовок содержит magic, версию, число сущностей/компонентов, tick/baseTick, размер payload, FNV-1a-сумму, сумму базы, хеш схемы, состояние RNG и поля мирового расширения v3. Сначала идут ECS-записи, поле за полем через Mirror; затем байты расширения для цепочек, ткани, воды и часов эмиттеров, адресуемые стабильными id подсистем, а не handles Box2D. Дельта-снапшот хранит только изменённые записи и всё равно содержит сумму базы, от которой он построен.

```
struct SnapshotHeader {
    char magic[8];              // FBSNAP03
    uint32 version;             // kSnapshotVersion
    uint32 flags;               // full or delta
    uint32 entityCount;
    uint32 componentCount;
    uint64 tick, baseTick;
    uint64 payloadBytes, checksum, baseChecksum, schemaHash;
    RngState rng;
    uint32 extensionBytes, extensionOffset;
    uint64 extensionChecksum;
};
```

Каждый зарегистрированный компонент даёт имя, стабильный type id, layout hash Mirror, канонический размер и четыре указателя на функции (has, capture, restore,

remove). Layout hash отклоняет изменение поля с тем же размером; schema hash отклоняет другой набор компонентов. Restore не выделяет память: арена снапшотов и таблицы сущностей фиксированы в Initialize().

6. Физика: Box2D v3, DOD-бэкенд, цепочки, ткань и вода

Публичные заголовки физики не включают box2d.h. RigidBody2D и Collider2D хранят упакованные b2BodyId/b2ShapeId в uint64_t, поэтому геймплей, scene IO и сетевой слой компилируются без C API. PhysicsSystem владеет миром; TransportGroups, VehicleWheels и системы соединений — явно знающие Box2D единицы трансляции, которые держат C API за теми же упакованными handles. Единицы заданы явно: Box2D работает в метрах, рендерер в пикселях, а PhysicsDesc::pixelsPerMeter — единственная точка преобразования.

Модель	**Детали**
Твёрдые тела	Мир Box2D v3, фиксированный шаг, события контактов, флаг телепорта
Верёвка / цепь / граф пружин	Граф соединений капсульных звеньев: цепь, верёвка и пружины с пределами прочности, связями и ремонтом
Ткань	Verlet-сетка 32x32, ограничения Гаусса-Зейделя, overlap-запрос к формам Box2D
Вода	Аналитические волны GerstnerField и ShallowWaterGrid на mdspan
DOD-бэкенд	SoA AABB-солвер с сортированной равномерной сеткой; без joints, body CCD и вращательного контактного отклика
Транспорт	Носители с путём/motion, контактной опорой и конвейерами; weld-джойнты Box2D при наличии
Колёса	Подвеска на wheel-джойнтах Box2D, момент и торможение по колёсам; без неявного powertrain

Верёвка — не специальное тело. Это цепочка небольших твёрдых звеньев, соединённых связями; каждая связь несёт свой материал (жёсткий distance, пружинный distance, revolute с лимитами, пружинный revolute), свои пределы прочности в ньютонах и ньютон-метрах, необязательное накопление повреждений и своё состояние ремонта. Поэтому цепь — это конструктор поверх фиксированного графа соединений: секции сращивают цепь и верёвку, связи соединяют два звена разных цепей или два обычных тела, разрезы из геймплея попадают в детерминированный журнал команд, а разорванная связь оставляет звенья живыми. Система жёстко ограничивает число цепочек, звеньев и соединений, выделяет пул один раз и ничего не аллоцирует на кадровом пути. Она владеет созданными звеньями и никогда — якорными сущностями; порядок Shutdown важен, потому что уничтожение мира Box2D инвалидирует все сохранённые id, а Update должен идти сразу после шага физики, чтобы забрать события суставов Box2D.

Ткань — Verlet mass-spring сетка, проецируемая фиксированным числом итераций Гаусса-Зейделя и затем выталкиваемая из существующих форм Box2D. Это не второй мир коллизий и не дублирование геометрии уровня. CPU пишет позиции MeshVertex;

рендерер передаёт их тем же деформируемым путём, что и скинированный меш, поэтому ткань не добавляет второй путь отрисовки. Закреплённая вершина — просто invMass = 0.

Две модели воды живут в одной системе. GerstnerField — небольшая сумма трохоидальных волн: детерминированная, аналитическая, без аллокаций, с якобианом для появления пены на гребнях. ShallowWaterGrid хранит поле высоты/скорости в mdspan-представлениях над фиксированными массивами и интегрирует линеаризованные уравнения мелкой воды с ограниченным числом CFL. Ни одна модель не знает о RHI; рендерер сэмплирует поле или загружает его представление высоты как R32F-текстуру.

DOD-бэкенд — вторая реализация того же контракта PhysicsSystem, выбираемая политикой проекта или PhysicsDesc.backend. Он держит позиции, предыдущие позиции, скорости, ускорения, повороты, обратные массы, gravity scale, размеры, флаги и счётчики сна в параллельных SoA-колонках и разрешает пары через сортированную равномерную сетку. У каждого тела свои gravity и time scale, замороженные тела не интегрируются, а контакты сообщают begin и end touch один раз на пару. PhysicsSystem::Capabilities() — единственный поддерживаемый запрос возможностей. Транспорт и колёса явно объявляют границу бэкенда: опора и конвейеры работают на обоих солверах, а weld- и wheel-джойнты требуют Box2D.

Симуляция не вызывает платформенный libm. `Engine/Core/SimMath.hpp` даёт `Sin`/`Cos`/`SinCos`, `Atan2`, `Acos`, `Cbrt`, `Exp`, `Log`, `Pow` и `WrapPi`, построенные на точном приведении аргумента и полиномах с fused multiply-add, где `ldexp` используется только для масштабирования экспоненты. Углы в трансформах, IK и транспорте, волны воды, спад гравитации, затухание, геймплейные множители, спад света и кривые ввода идут через этот слой, поэтому checksum'ы rollback совпадают на Clang и GCC на всех поддерживаемых целях. Box2D сохраняет свой floating-point путь, поэтому соединения и CCD остаются за general-purpose-солвером.

7. Сетевой роллбэк и Snapshot v4

NetRollbackSession связывает Snapshot v4, CommandLog, Pcg32 и геймплейные компоненты (теги, атрибуты, эффекты и скиллы). Сессия владеет фиксированным кольцом полных снапшотов (один на симулированный тик), кольцом ввода на игрока, картой стабильных NetEntityId в локальные EnTT id и циклом повтора. Всё после Initialize() не аллоцирует память: арена снапшотов, таблицы сущностей, кольца ввода и метаданные кадров имеют фиксированную ёмкость. Колбэк симуляции — указатель на функцию плюс void*, а не std::function.

Когда удалённый ввод приходит с опозданием, сессия восстанавливает снапшот непосредственно перед самым ранним исправленным тиком, восстанавливает RNG и курсор CommandLog из того же слота, повторно применяет структурные команды, ресимулирует затронутые тики с исправленным вводом и предсказанием для всё ещё отсутствующих входов, снимает новые снапшоты и сообщает, какие кадры реально изменились.

Константа	Детали
kMaxNetPlayers = 8	Ёмкость кольца ввода и массива входов тика
kMaxRollbackTicks = 256	Глубина истории снапшотов для коррекции
kMaxNetEntities = 4096	Таблица стабильных сетевых сущностей
kInputRingCapacity = 256	Окно перезаписи опоздавшего ввода

NetInputFrame занимает 24 байта и использует знаковые 16-битные fixed-point оси, а не float: битовый шаблон float не гарантированно переживает другой компилятор, а симуляция должна быть чистой функцией входных битов. Статус кадра каждого игрока — Confirmed, Predicted или Missing. Опоздавший ввод перезаписывает предсказание на месте в том же тике, что и требуется для повтора.

NetEntityId назначается авторитетом (или локальным пиром для локально принадлежащих объектов) и никогда не выводится из EnTT id. Восстановленный снапшот, другой порядок спавна или другой аллокатор не должны его менять. Реплицируемое геймплейное состояние — компактная маска тегов плюс фиксированные записи атрибутов, эффектов и скиллов; контрольная сумма сворачивает каждую из них на сущность. CommandLog фиксирует структурные spawn/despawn/add/remove, поэтому роллбэк повторяет и структуру, и значения.

8. Пайплайн ассетов и инфраструктура редактора

Компонент никогда не хранит путь. Он хранит небольшой AssetHandle с проверкой поколения, указывающий на строку таблицы метаданных AssetDatabase. Строка владеет стабильными AssetId/GUID, видом ресурса, исходным и приготовленным путями, списком зависимостей и настройками импорта. AssetHandle::Generation обнаруживает переиспользованный слот; AssetMeta::contentGeneration позволяет рендереру или кешу сравнивать версии содержимого. Горячая перезагрузка никогда не показывает наполовину обновлённый ассет: правки готовятся и фиксируются атомарно.

```
[[asset]]
guid = "018f0c2e-..."
type = "texture"
source = "atlas/orb.png"
cooked = "atlas/orb.ftex"
format = "bc7_srgb"
depends = ["atlas/atlas.toml"]

[import]
generate_mips = true
max_size = 2048
```

AssetDatabaseWatchSystem следит за исходными папками через OS API и складывает изменённые пути в фиксированное SPSC-кольцо. Поток watcher никогда не меняет базу или RHI; Poll() выполняется в потоке редактора, находит AssetHandle, планирует кукинг, готовит новые метаданные, фиксирует их, увеличивает поколение содержимого и рассылает уведомления о перезагрузке. Свободный PNG/WAV/WebM без

строки .femeta импортируется автоматически с детерминированными id и настройками по умолчанию.

CommandStack — движок undo/redo редактора. Правка инспектора или перетаскивание ImGuizmo — одна транзакция: BeginTransaction, Record одной или нескольких сущностей, Commit. Записанный компонент хранится как разреженная байтовая дельта, поэтому перемещение одного Vector2D в 20-килобайтном SkeletonComponent стоит десятки байт. Хранилище — фиксированный массив команд плюс одна арена payload; никакого operator new, std::function и исключений на пути правки. Стек существует только в редакторе и не линкуется в игру.

Инструмент	Детали
VFS	Монтирует архивы .fempak и свободные каталоги в одно пространство путей
VFS	Детерминированный порядок чтения, range-чтение, без аллокаций на файл в горячем пути
fby_fempak	Кукер текстур/аудио/локализации; байт-в-байт совпадает с импортёром редактора
fby_loc_tool	TOML локализации -> пакет FlatBuffers
fby_scene_tool	Компилятор сцен/префабов и проверка round-trip
fby_aotc	Байткод -> C++26 -> нативный плагин во время кукинга

9. Визуальные графы и AOT-транспилятор

Material Graph — небольшой DAG в трёх плоских массивах фиксированной ёмкости: узлы, пины и параметры. Целый материал — одно trivially copyable значение, которое можно memcpy-ить, снимать в undo-системе и хешировать без аллокатора. Компилятор проверяет ровно один выходной узел, подключённость обязательных пинов, совместимость типов и отсутствие циклов, затем генерирует одну GLSL-временную переменную на узел. Никакого интерпретатора и диспатча на кадр: материал стоит ровно столько, сколько стоит его сгенерированный шейдер. Хеш не включает UI-позиции, поэтому перетаскивание узла не перекомпилирует шейдер.

Ограничение	Детали
MaterialGraph nodes = 128	Больший материал — признак проблемы дизайна, а не повод растить рантайм
MaterialGraph pins = 384	Каждый входной слот каждого узла, непрерывно по узлу
MaterialGraph params = 64	Payload скаляра, vec2, vec3, цвета, слоя атласа и texture sample
LogicGraph nodes = 128	Документ редактора — один POD на 10 КиБ
LogicGraph links = 256	Разреженная таблица связей; одна транзакция на правку ребра

NodeCanvasPanel — транзакционный холст ImNodes. Панель предоставляет таблицу CanvasModelOps из обычных указателей на функции и void*; холст не видит виртуального интерфейса. Холст владеет только состоянием взаимодействия и открывает ровно одну транзакцию CommandStack на перетаскивание или правку

ребра. Поскольку весь граф зарегистрирован как один внешний буфер, перемещение узла записывает 8-байтный Vector2D, а не весь граф на 7 КиБ.

VisualScript компилирует LogicGraph в канонический байткод с топологической проходкой Кана; цикл — ошибка компиляции, а не переполнение стека в рантайме. VM исполняет ровно тот артефакт, который поставляет кукер. Для релиза AotTranspiler генерирует одну плоскую единицу трансляции C++26 с одной функцией на событие и без зависимостей от движка, кроме AotAbi.hpp; AotBuilder запускает clang/g++ -O3 -shared в фоновом jthread, а редактор опрашивает его состояние, не блокируя кадр. Сгенерированный модуль несёт хеш графа и каждый LayoutHash компонента; загрузчик проверяет их до первого вызова.

```
LogicGraph -> GraphCompiler -> байткод -> VM (PIE)
                        \-> AotTranspiler -> C++26 -> clang/g++ -O3 -shared -> плагин

MaterialGraph -> проверка типов -> GLSL -> glslang -> SPIR-V -> VkPipeline
```

10. Контроль качества и верификация

Release candidate проверяется набором CTest: rollback replay, RNG/noise, сетевой роллбэк, indirect mesh smoke, script VM, script AOT, PinkUI, DSP mixer, audio cook, material graph, video, texture cook, метаданные ассетов, animation timeline, editor command stack, инфраструктура редактора, graph editor, расширения физики, доступность UI, P3 integration и контракт Definitions. Тесты — обычные исполняемые файлы без фреймворка, поэтому работают headless и на консольном devkit.

Ворота	**Детали**
ctest	полный набор проходит; GPU не требуется для headless-подмножества
CheckStyle.py	PascalCase-имена, требуемое дерево, отсутствие Vec2/3/4, RTTI-токенов и кириллицы, Allman, скобки
clang-format	Allman, 4 пробела, 120 колонок, обязательные скобки
clang-tidy	Параметры In*, выходы Out*, локальные Scoped*, без затенения типов
RTTI scan	dynamic_cast/typeid запрещены в исходниках; код движка не использует информацию типов компилятора

Style gate намеренно проверяет репозиторий, а не только diff. Новый файл с именем в нижнем регистре, алиасом Vec2, комментарием в конце строки или кириллицей падает на ревью до попадания в main. Комментарии только на английском, одной строкой, над кодом и без точки в конце; единственные оставшиеся комментарии в конце строки — два подавления NOLINT, требуемых clang-tidy.

```
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=RelWithDebInfo -DFBY_BUILD_TESTS=ON
cmake --build build --parallel 8
ctest --test-dir build --output-on-failure
python Source/Tools/Dev/CheckStyle.py
```

11. Локализация и документация

Этот документ генерируется Documentation/Tools/GenerateDocumentation.py. Скрипт владеет одной языконейтральной моделью контента и рендерит независимые английское и русское издания в Markdown, Word и PDF. У каждого издания собственный DOCX и PDF; PDF экспортируется из DOCX соответствующего языка через LibreOffice Kit.

Стили не зависят от языка: FBY Title, FBY Subtitle, FBY Heading 1-3, FBY Body, FBY Code, FBY Caption, FBY Table Header, FBY Table Body и FBY Note. Каждый run несёт явный тег w:lang (en-US или ru-RU), а стили тела и заголовков задают шрифты ascii, hAnsi, eastAsia и cs. Добавление языка означает добавление колонки перевода и новой секции; переписывать стили или делать отдельную вёрстку не нужно.

Артефакт	Детали
FemboyEngineArchitecture.md	Объединённый документ с обеими языковыми частями
FemboyEngineArchitecture_EN.md	Отдельный английский Markdown
FemboyEngineArchitecture_RU.md	Отдельный русский Markdown
FemboyEngineArchitecture.docx	Документ Word с двумя независимыми секциями
FemboyEngineArchitecture.pdf	PDF, экспортированный из документа Word

```
python Documentation/Tools/GenerateDocumentation.py
python Documentation/Tools/GenerateDocumentation.py --skip-pdf
python Documentation/Tools/GenerateDocumentation.py --node <node> --lo-cli
<libreoffice-kit cli.js>
```

12. Следующие шаги после Release

- Сделать CheckStyle.py тестом CI и pre-commit-хуком, чтобы файл в нижнем регистре или алиас Vec2 не могли попасть в main.
- Добавить политику стабильности публичного C++ API: замороженные layout компонентов, тесты совместимости Snapshot v4 и матрицу ABI-версий для плагинов.
- Завершить паритет Metal: indirect-путь мешей, compute-скиннинг и тот же контракт FramePacket, что у Vulkan, затем включить CI macOS/iOS.
- Добавить сжатие дельта-снапшотов и бюджет пропускной способности, а также просмотрщик повторов, который шагает по кольцу снапшотов и накладывает исправленные кадры.
- Перенести ткань и мелкую воду на опциональный GPU-солвер, сохранив CPU Verlet как детерминированный fallback для роллбэка.
- Добавить представление графа зависимостей в базу ассетов с обнаружением циклов и экспортом порядка кукинга.
- Добавить локализацию редактора: ту же систему стилей с языковыми тегами для строк панелей и горячую перезагрузку из каталога .fbin.
- Добавить детерминированный replay/fuzz-стенд в матрицу CI и бюджет регрессии производительности кадрового пути.

A. Инварианты и ёмкости

Инвариант	Детали
RenderItem	48 байт, trivially copyable, фиксированный GPU-layout
MeshVertex / SkinnedVertex	40 / 32 байта, совместимы со std430
SnapshotHeader	104 байта, little-endian, контрольная сумма FNV-1a
Узлы SystemGraph	64, ёмкость на этапе компиляции, однократная топологическая сортировка
Сетевые игроки / тики / сущности	8 / 256 / 4096, фиксированные кольца и таблицы
Узлы / пины / параметры MaterialGraph	128 / 384 / 64, плоские массивы
Узлы / связи LogicGraph	128 / 256, один POD-документ на 10 КиБ
Команды / payload CommandStack	512 команд, арена payload 4 МиБ по умолчанию

B. Глоссарий

Термин	Детали
DOD	Data-oriented design: данные плоские, горячие пути без аллокаций и дружелюбны к кешу
MDI	Multi-draw indirect: одна indexed indirect-команда несёт много отрисовок на инстанс
LBS	Linear-blend skinning: palette[bone] * bindPosition за один compute-проход
.femeta	Строка метаданных ассета: стабильный id, исходный/приготовленный пути, зависимости, настройки импорта
AOT	Ahead-of-time: байткод, превращённый в нативную разделяемую библиотеку во время кукинга
PIE	Play-in-editor: редактор запускает игровую симуляцию в собственном вьюпорте